

Architectural Refinement and Fault Tolerance for the BreakDex "Add Move Flow"

Deconstruction of the "Add Move Flow" as a Categorical System

A rigorous analysis of the "Add Move Flow" requires a formal model that transcends simple bug reports and addresses the system's underlying architectural structure. By employing principles from category theory, the system can be modeled as a collection of objects, representing states and data structures, and morphisms, representing the functions and transformations that operate upon them. This categorical approach allows for a precise deconstruction of data flow, composition of operations, and the identification of architectural flaws, such as non-commutative diagrams that lead to inconsistent states. The systemic failures reported—memory leaks, data corruption, and inconsistent application states—are not isolated incidents but rather emergent properties of a flawed composition of these fundamental components.

System Objects: The State-Holders of the Flow

The objects within the "Add Move Flow" category are the distinct data structures and state-holders that define the system at any given point in its execution. Each object represents a well-defined piece of information or a component responsible for managing a part of the application's state.

- **User-Selected Asset (PHAsset):** This is the initial object in the flow, representing the immutable reference to a video asset within the user's PHPhotoLibrary. A PHAsset object contains metadata about the media but not the media data itself. It serves as the entry point for all subsequent video processing operations.

- **Video Player State (AVPlayer, AVAsset):** Upon selection, the PHAsset is resolved into an AVAsset, which is an object that models the timing and media data of the asset. The AVAsset is then used to initialize an AVPlayer instance. This composite object represents the video in a playable state, forming the core of the interactive trimming view. It encapsulates the video's timeline, tracks, and presentation state.
- **Trimmer View Model (TrimmerViewModel):** This is a critical state-holding object that serves as the central hub for the trimming and naming UI. It encapsulates the AVAsset, the user-defined trim range (represented as a CMTimeRange), the selected rotation angle (e.g., as a CGFloat representing degrees), and the current UI state. As identified in the provided logs, this object is the locus of a severe memory management failure, making its lifecycle and dependencies a primary subject of this investigation.
- **Trimmed Video Data (URL):** This object is the product of the video export process. It is a temporary file URL pointing to the new video file created by AVAssetExportSession. This file contains the video data precisely trimmed and rotated according to the state held by the TrimmerViewModel. The lifecycle of this temporary file must be carefully managed to avoid consuming excess disk space.
- **Photo Library Service (PhotoKitService):** This object represents an architectural component designed to encapsulate all interactions with the PHPhotoLibrary framework. It is responsible for abstracting the complexities of finding or creating photo albums and saving new media assets. Its design and implementation are crucial for ensuring atomic and idempotent interactions with the user's photo library.
- **CoreData Move Entity (Move):** This is the terminal object in the persistence pipeline. It represents a single "move" record within the application's CoreData database. This entity must contain the user-provided name for the move and a durable, persistent reference to the video asset that has been saved to the PHPhotoLibrary. The nature of this reference is a key architectural decision point for ensuring long-term data resilience.

System Morphisms: The Transformations and Actions

The morphisms are the functions and processes that transform one object into another or trigger state changes within them. They represent the "arrows" that connect the objects in the system's category, defining the flow of data and control.

- **selectVideo: PHAsset → TrimmerViewModel:** This morphism represents the function that initiates the flow. It takes the initial PHAsset object and is responsible for creating and configuring the TrimmerViewModel. This process involves requesting the underlying AVAsset from the PHPhotoLibrary and preparing the AVPlayer for presentation.
- **applyTrim: (TrimmerViewModel, CMTimeRange) → TrimmerViewModel:** This morphism models the user's interaction with the trimming interface. It takes the current TrimmerViewModel and a new CMTimeRange as input and produces an updated

TrimmerViewModel with the new trim selection. This is a pure state transformation within the view model. A similar morphism, `applyRotation`, handles changes to the video's orientation.

- **exportVideo: TrimmerViewModel → Task<URL, Error>:** This is a computationally intensive and asynchronous morphism. It takes the final state of the TrimmerViewModel (including the source asset, trim range, and rotation) and initiates an AVAssetExportSession. The output is a Task that will eventually produce either a URL object pointing to the temporary trimmed video file or an Error if the export fails.
- **saveToPhotos: (URL, AlbumName) → Task<String, Error>:** This morphism, handled by the PhotoKitService, takes the temporary video file URL from the exportVideo morphism and a target album name. It orchestrates the interaction with PHPhotoLibrary to save the video into the specified album, creating the album if it does not exist. Upon success, it returns a String containing the localIdentifier of the newly created PHAsset, which is a unique and persistent identifier for the asset within the user's library.¹
- **persistToCoreData: (MoveName, assetIdentifier: String) → Move:** This is the final morphism in the flow. It takes the user-provided name for the move and the localIdentifier returned by saveToPhotos and creates a new Move entity in the CoreData persistent store, completing the end-to-end process.

Analysis of Morphism Composition: Identifying Non-Commutative Diagrams

The entire "Add Move Flow" can be modeled as a single composite morphism:

`AddMoveFlow = persistToCoreData ∘ saveToPhotos ∘ exportVideo ∘ applyTrim ∘ selectVideo`

In an ideal, error-free system, this composition would be atomic and transactional. A failure at any point would result in a complete rollback to the initial state, leaving no side effects or corrupted data. However, the observed behaviors indicate that the current implementation is neither atomic nor fault-tolerant. The composition of these morphisms does not commute with failure states, leading to inconsistencies.

For example, consider the `saveToPhotos` morphism. It is itself a composition of two sub-morphisms: `createAlbumIfNeeded` and `addAssetToAlbum`. The ideal system requires that the following diagram commutes: starting from a state with a video URL, the path of successfully creating an album and then successfully adding the asset should lead to the same final state as an atomic operation that does both.

However, the system can crash after `createAlbumIfNeeded` completes but before `addAssetToAlbum` begins. In this scenario, the system transitions to a state that is not on the ideal path—a state where an empty album exists. This is a classic example of a

non-commutative diagram in the context of system behavior. The operation (createAlbumIfNeeded; addAssetToAlbum) is not equivalent to atomicSaveToAlbum in the presence of failures.

This reveals a fundamental architectural deficiency: the system lacks transactional integrity. The "Add Move Flow" is a multi-stage pipeline involving several asynchronous, fallible operations that span different system frameworks (AVFoundation, Photos, CoreData). The current implementation treats this pipeline as a simple, linear sequence of commands. It lacks a unifying state management strategy or a robust error recovery mechanism. A failure at any stage does not trigger a rollback or cleanup of artifacts created in previous stages. This results in orphaned data, such as empty photo albums, and a corrupted overall system state. The solution, therefore, is not merely to fix individual bugs but to re-architect the flow to be transactional, ensuring that the entire composition of morphisms behaves as a single, indivisible unit of work.

Root Cause Analysis of Systemic Failures

The reported issues—memory leaks, crashes, empty albums, performance degradation, and data integrity violations—are direct consequences of the architectural flaws identified in the categorical model. A detailed analysis of the provided diagnostic information, when viewed through this structural lens, reveals the precise root causes of each failure.

The Memory Leak Functor: Mapping Ideal State to a Dangling Reference

The provided crash log is the most critical piece of evidence. It points to a severe memory management issue that is the likely trigger for several other downstream problems, including the creation of empty albums.

- **Log Analysis and Interpretation:** The key log message, Object 0x11ce10880 of class TrimmerViewModel deallocated with non-zero retain count 2, is a diagnostic message from the Objective-C runtime. It is crucial to interpret this message correctly. It does not mean that dealloc is being called on an object that is still actively and correctly referenced. Instead, it indicates that the memory for the object is being reclaimed (likely because the scope that owned it, such as a view controller, has ended), but its internal retain count is unexpectedly greater than one. This points directly to a memory management anomaly, most commonly a retain cycle.² A retain count of

2 is the classic signature of two objects holding strong references to each other, preventing either from being deallocated by Automatic Reference Counting (ARC).⁵

- **Hypothesis - The Retain Cycle:** The most probable cause is a strong reference cycle involving a closure. This is a frequent issue in Swift, particularly with view models that perform asynchronous work or interact with services via callbacks.⁷ The cycle likely forms as follows:
 1. The TrimmerViewModel (or an object it owns) holds a strong reference to a closure, for example, as a completion handler for a task.
 2. This closure needs to access a method or property on the TrimmerViewModel itself. To do this, it "captures" self.
 3. By default, closures capture variables with strong references.
 4. This creates a cycle: TrimmerViewModel has a strong reference to the closure, and the closure has a strong reference back to TrimmerViewModel.

When the view associated with the TrimmerViewModel is dismissed, the navigation controller or parent view releases its strong reference to the view model. However, because of the cycle, the retain count never drops to zero. The TrimmerViewModel and the closure it holds are now "leaked"—they remain in memory but are inaccessible to the rest of the application.⁶

- **The Crash Isomorphism: `__pthread_kill` as an Equivalence to State Corruption:** The crash itself, culminating in a `__pthread_kill` signal on Thread 1, is the observable, catastrophic result of the memory leak. While the TrimmerViewModel object is leaked, other parts of the system may proceed as if it were properly deallocated. Subsequent attempts to interact with this "zombie" object, or general memory pressure causing the operating system to reclaim memory in an inconsistent state, can lead to heap corruption. The runtime environment detects this unrecoverable state and terminates the application process via a POSIX signal (SIGABRT, which is what `pthread_kill` often triggers) to prevent further damage. The crash can be seen as isomorphic to the memory leak; they are distinct phenomena (a memory management error versus a process termination), but they are structurally equivalent representations of the same fundamental failure of state integrity.

Idempotency Failure in PhotoKit Interactions: The "Empty Album" Anomaly

The appearance of empty "BreakDex" albums in the user's Photos app is a direct consequence of the system's lack of atomicity, a problem that is exacerbated by the memory leak and crash.

- **Problem Analysis and Causal Relationship:** Saving a video to a custom album is a two-step process within the Photos framework: first, one must find or create the album

(PHAssetCollection), and second, one must add the asset (PHAsset) to that collection.¹²

These are typically performed as two separate requests within a `PHPhotoLibrary.performChanges` block. The memory leak and subsequent crash provide a clear causal mechanism for the empty album anomaly. The application successfully executes the first step (creating the album via `PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle:)`), which is committed to the Photos library. However, the application then crashes before it can execute or complete the second step of adding the video asset. The result is an orphaned, empty album—a persistent artifact of a failed transaction.

- **Lack of Idempotency:** Beyond the crash-related failure, the logic for creating the album is likely not idempotent. An idempotent operation is one that can be applied multiple times without changing the result beyond the initial application. In this context, a function `getOrCreateAlbum(name: "BreakDex")` should result in exactly one album named "BreakDex," regardless of how many times it is called, even concurrently. If the current implementation's check for the album's existence is flawed, or if it lacks proper synchronization, it's possible for multiple, near-simultaneous save attempts to each conclude that the album does not exist, leading to multiple creation requests and duplicate albums.¹ A robust implementation must fetch collections with a specific predicate and only proceed with creation if the fetch result is empty, all while handling the asynchronous nature of Photos framework interactions.

Performance Bottlenecks as Inefficient Functors: The "Long Save" Problem

The user's perception that the saving process "take[s] that long" points to significant performance issues, likely stemming from both the nature of the work being done and how it is being executed.

- **AVFoundation Analysis:** The `AVAssetExportSession` is the core of the video processing pipeline. Its performance is highly dependent on its configuration.
 1. **High-Quality Preset:** Using the `AVAssetExportPresetHighestQuality` preset instructs the framework to perform a full re-encode of the video file. This process is extremely CPU and I/O intensive, and its duration scales with the video's resolution, bitrate, and length. Exporting a 4K video, even a short one, can easily take several seconds on a mobile device.¹⁷
 2. **Main Thread Blocking:** The most critical performance anti-pattern is performing this intensive work on the main thread. If the `exportAsynchronously(completionHandler:)` method is called and its completion handler, or any subsequent processing like saving to the photo library, performs significant work on the main queue, the application's user interface will become

completely unresponsive. This "freeze" is what the user perceives as an unacceptably long save time.¹⁸ The entire export and save process must be dispatched to a background queue to keep the UI responsive.

- **UX Enhancement Request:** The user's specific requests for an elapsed time indicator and a more minimal loading UI are direct and insightful responses to this poor performance. An elapsed time counter, implemented with a Timer in SwiftUI, provides crucial feedback that the app has not stalled and helps the user gauge the wait time, improving perceived performance.¹⁹ Replacing a blocking, full-screen loading bar with a subtle, non-blocking indicator (like a small spinner) allows the user to understand that a background task is in progress without preventing them from potentially navigating away or interacting with other parts of the app.²²

Data Integrity Failure: The WYSIWYG Violation

The requirement that "The assets stored should always preserve the exact trimming range + rotation from the NameMoveView" indicates a failure in the data transformation logic. The final, persisted asset does not match the user's configured preview—a violation of the "What You See Is What You Get" (WYSIWYG) principle.

- **Root Cause:** This failure originates within the exportVideo morphism, specifically in the configuration of the AVAssetExportSession and its associated AVMutableVideoComposition.
 - **Rotation:** Video assets have inherent orientation metadata. To apply an additional user-selected rotation (e.g., 90, 180, 270 degrees), a CGAffineTransform must be correctly calculated and applied to the video track via an AVMutableVideoCompositionLayerInstruction. This involves composing the original transform of the video track with the new rotation transform. An error in this matrix arithmetic will result in a video saved with the wrong orientation.
 - **Trimming:** The AVAssetExportSession has a timeRange property. This property must be set precisely to the CMTimeRange value stored in the TrimmerViewModel. If this property is not set, the entire video will be exported. If it is set incorrectly, the final trim will not match the user's selection.

The combination of these failures paints a picture of an application feature that is architecturally fragile, prone to catastrophic failure, and provides a poor user experience. The solutions must be equally comprehensive, addressing not just the surface-level bugs but the underlying structural weaknesses.

A Refined Architecture for a Fault-Tolerant Flow

To address the systemic failures identified, a revised architecture is necessary. This new architecture will be designed for robustness, transactional integrity, and performance. The required changes can be conceptualized as a series of "natural transformations"—principled refinements that map the flawed implementation to an ideal, fault-tolerant system without altering the fundamental business logic.

Achieving State Integrity and Lifecycle Correctness

The cornerstone of the new architecture is a robust state management system that ensures data integrity and correct object lifecycles, starting with the resolution of the critical memory leak.

- **Introducing a LoadableObject State Machine:** The entire "Add Move Flow" will be governed by a central ObservableObject. This object will implement a state machine pattern, similar to the LoadableObject concept, to make the flow's state explicit and manageable.²³ The state will be represented by a generic enum, FlowState, which will clearly define each stage of the process:

```
Swift
enum FlowState<Success, ProgressValue> {
    case idle
    case loadingVideo
    case readyForTrimming(AVAsset)
    case exporting(ProgressValue)
    case savingToLibrary
    case failed(Error)
    case success(Success)
}
```

The SwiftUI view layer will observe this state object. UI elements, such as loading indicators and enabled/disabled buttons, will be a direct function of the current state. This eliminates implicit state management and makes the flow's behavior predictable and debuggable.

- **Resolving the Retain Cycle (Natural Transformation):** The memory leak in the TrimmerViewModel must be resolved to ensure its lifecycle is correctly managed by ARC. The transformation is a direct application of Swift's memory management rules for closures. The closure that captures self (the TrimmerViewModel) must be modified to use

a weak capture list. This breaks the strong reference cycle.⁵

The fix involves changing the closure's capture list from an implicit strong capture to an explicit weak one:

Swift

```
// Before: Strong capture creating a retain cycle
```

```
someService.performAsyncTask {  
    self.updateUI()  
}
```

```
// After: Weak capture breaking the cycle
```

```
someService.performAsyncTask { [weak self] in  
    guard let self = self else { return }  
    self.updateUI()  
}
```

This change is a natural transformation that corrects the morphism causing the leak. The `guard let self = self` pattern is used to create a temporary strong reference to `self` for the duration of the closure's execution, ensuring the object is not deallocated mid-operation while preventing a permanent retain cycle.²⁵

- **Defining the Source of Truth: CoreData as a Pointer to Photos Library:** A profound architectural shift is required to achieve long-term data resilience, as requested by the user's directive that the "BreakDex" album be the "absolute source of asset truth." This principle mandates a decoupling of the application's metadata from the media data itself. The application's database should not store fragile file paths or, worse, the video data itself. Instead, it should store a durable, permanent link to the asset as it is managed by the operating system's Photos framework.

The PHAsset object's `localIdentifier` property provides exactly this mechanism. It is a unique, persistent string that identifies the asset across device restarts and even system updates. By storing this identifier, the application's CoreData store becomes a lightweight index that points to the authoritative media assets in the PHPhotoLibrary.

This design offers significant benefits:

1. **Robustness:** If the application is deleted and reinstalled, or if its local cache is cleared, it can use the stored `localIdentifier` to re-fetch the PHAsset and re-establish the link to the video.
2. **Data Integrity:** It respects that the user manages their photos and videos through the Photos app. Deletions or edits made there are the source of truth.
3. **Future-Proofing:** This architecture makes the proposed "re-link" functionality straightforward to implement.

The Move entity in the CoreData model will be modified to include `name: String` and `assetIdentifier: String`. The `saveToPhotos` morphism will now be responsible for returning this `assetIdentifier` upon a successful save.

An Idempotent and Atomic PhotoKitService

All interactions with the PHPhotoLibrary will be centralized into a dedicated PhotoKitService singleton. This service will be designed to provide an idempotent and atomic API for album and asset management, preventing the creation of duplicate or empty albums.

- **getOrCreateAlbum(named:) Morphism:** This function will be implemented to be fully idempotent and safe from race conditions. It will be an async function that first performs a fetch request to the PHPhotoLibrary to find an existing album with the specified name.

Swift

// Simplified logic for the idempotent album fetch/create

```
func getOrCreateAlbum(named name: String) async throws -> PHAssetCollection {
```

```
    let fetchOptions = PHFetchOptions()
```

```
    fetchOptions.predicate = NSPredicate(format: "title = %@", name)
```

```
    let collections = PHAssetCollection.fetchAssetCollections(with:.album, subtype:.any, options: fetchOptions)
```

```
    if let existingAlbum = collections.firstObject {
```

```
        return existingAlbum
```

```
    }
```

// Album does not exist, create it.

```
    var placeholder: PHObjectPlaceholder?
```

```
    try await PHPhotoLibrary.shared().performChanges {
```

```
        let request =
```

```
PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle: name)
```

```
        placeholder = request.placeholderForCreatedAssetCollection
```

```
    }
```

```
    guard let albumIdentifier = placeholder?.localIdentifier else {
```

```
        throw PhotoKitError.albumCreationFailed
```

```
    }
```

// Fetch the newly created album to return it

```
    let newCollections = PHAssetCollection.fetchAssetCollections(withLocalIdentifiers: [albumIdentifier], options: nil)
```

```
    guard let newAlbum = newCollections.firstObject else {
```

```
        throw PhotoKitError.albumNotFoundAfterCreation
```

```

    }
    return newAlbum
}

```

This logic ensures that a creation request is only performed if and only if no album with the target name exists.¹ The use of `async/await` helps manage the asynchronous nature of `performChanges`.

- **saveVideo(at:toAlbum:) Morphism:** This function will orchestrate the entire save operation as a single, atomic transaction to prevent the "empty album" anomaly.
 1. It will first call the `getOrCreateAlbum(named:)` function to obtain a reference to the target `PHAssetCollection`.
 2. It will then initiate a *single* `PHPhotoLibrary.performChanges` block.
 3. Inside this block, it will create a `PHAssetChangeRequest` from the temporary video URL.
 4. It will then create a `PHAssetCollectionChangeRequest` for the target album.
 5. Crucially, it will use the `placeholderForCreatedAsset` from the asset creation request and pass it to the `addAssets(_:)` method of the album change request.¹² This links the two operations within the same change transaction.
 6. The completion handler of `performChanges` will capture the `localIdentifier` from the asset's placeholder and return it. This identifier is then passed to the CoreData persistence layer. This ensures that an album is never created without its corresponding asset being added in the same atomic operation.

Optimization via Adjoint Functors: Offloading Intensive Work

Performance will be addressed by moving all computationally expensive operations off the main thread and optimizing the export process. This can be viewed as applying an adjoint functor—a structure-preserving mapping that finds a more efficient representation (a background task) for a computationally expensive problem.

- **Asynchronous Video Export:** The entire `AVAssetExportSession` process will be encapsulated in an `async throws` function. This function will be explicitly run on a global background `DispatchQueue` to ensure the main thread and UI remain fully responsive.

Swift

```

func export(asset: AVAsset, timeRange: CMTimeRange, rotation: CGAffineTransform) async
throws -> URL {
    //... setup export session...

    // Set a lower, visually lossless preset if acceptable
    // session.presetName = AVAssetExportPreset1920x1080

```

```

    session.timeRange = timeRange
    session.videoComposition = createVideoComposition(for: asset, with: rotation)
    session.shouldOptimizeForNetworkUse = true

    // Await the asynchronous export
    await session.export()

    if session.status == .completed {
        return session.outputURL!
    } else {
        throw session.error ?? ExportError.unknown
    }
}

```

The progress of the export session can be monitored using a Timer or Key-Value Observing on the session's progress property. This progress value will be published back to the main actor to update the state machine (e.g., transitioning the state to `.exporting(session.progress)`), which in turn updates the UI. Investigating visually lossless presets like `AVAssetExportPreset1920x1080` instead of `AVAssetExportPresetHighestQuality` could offer a significant performance improvement with negligible impact on quality for most use cases.¹⁷

- **Correctly Applying Transformations:** To fix the WYSIWYG violation, the logic for creating the `AVMutableVideoComposition` will be carefully reviewed and corrected. The `AVMutableVideoCompositionLayerInstruction` must be configured with a `CGAffineTransform` that correctly composes the video's original preferred transform with the user-selected rotation. The `timeRange` property of the `AVAssetExportSession` will be set directly from the `TrimmerViewModel`'s state to ensure the trim is precise.

Enhancing User Experience Morphisms

The user interface will be updated to provide clear, non-intrusive feedback during long-running operations, directly addressing the user's requests.

- **Minimal Loading Indicator:** Once the user confirms the trim and name, the UI state will transition to `.exporting`. Instead of a blocking progress bar, a more subtle, non-blocking indicator will be displayed. A standard `ProgressView()` in SwiftUI is an excellent choice for this, as it is lightweight and conforms to platform conventions.²² It can be shown conditionally based on the state machine.

Swift

```
// In the SwiftUI View body
if case.exporting = viewModel.state {
    ProgressView()
        .progressViewStyle(.circular)
}
```

This provides feedback without preventing the user from, for example, navigating back if they wish to cancel the operation.

- **Elapsed Time Gauge:** To provide a sense of progress during the potentially long export and save process, a simple elapsed time counter will be implemented. A `Timer.publish` publisher can be attached to the view, and its output will be used to update a `@State` variable every second.

```
Swift
@State private var elapsedTime: TimeInterval = 0
let timer = Timer.publish(every: 1, on:.main, in:.common).autoconnect()

//... in the view body
if case.exporting = viewModel.state {
    Text("Saving... \("\(formatTime(elapsedTime))")")
        .onReceive(timer) { _ in
            elapsedTime += 1
        }
}
```

This simple addition provides valuable reassurance to the user that the application is still working, transforming a frustrating wait into a transparent process.²⁰

Implementation, Verification, and Risk Mitigation

This final section outlines a concrete plan for implementing the proposed architectural changes, a strategy for verifying their correctness, and an analysis of potential risks and future considerations.

Targeted Code Refinements (with Snippets)

The following code snippets illustrate the key implementation details for the proposed

solutions.

- **TrimmerViewModel Retain Cycle Fix:**

Swift

// In TrimmerViewModel, where an asynchronous callback is used

```
private var cancellable: AnyCancellable?

func performSomeAsyncWork() {
    cancellable = somePublisher
        .sink(receiveCompletion: { _ in },
              receiveValue: { [weak self] value in
                  // Use a guard to safely unwrap the weak reference to self.
                  // This breaks the retain cycle.
                  guard let self = self else { return }
                  self.handleAsyncResult(value)
              })
}

func handleAsyncResult(_ result: String) {
    // Update view model state
}

deinit {
    // This print statement is a crucial debugging tool.
    // It should appear in the console when the view is dismissed.
    print("TrimmerViewModel deinitialized")
}
```

- **PhotoKitService Atomic Save Implementation:**

Swift

import Photos

```
actor PhotoKitService {
    //... (idempotent getOrCreateAlbum function as shown previously)...

    func saveVideo(at fileURL: URL, toAlbumNamed albumName: String) async throws -> String {
        let album = try await getOrCreateAlbum(named: albumName)

        var assetPlaceholder: PHObjectPlaceholder?

        try await PHPhotoLibrary.shared().performChanges {
            // 1. Create the asset creation request
```

```

        guard let assetRequest =
PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL: fileURL) else {
            // In a real implementation, throw a specific error
            return
        }

        // 2. Get the placeholder for the new asset
        assetPlaceholder = assetRequest.placeholderForCreatedAsset

        // 3. Create the album change request
        guard let albumChangeRequest = PHAssetCollectionChangeRequest(for: album) else {
            return
        }

        // 4. Add the asset placeholder to the album change request
        // The enumeration must be an NSArray
        let assets: NSArray = [assetPlaceholder!]
        albumChangeRequest.addAssets(assets)
    }

    guard let localIdentifier = assetPlaceholder?.localIdentifier else {
        throw PhotoKitError.assetIdentifierUnavailable
    }

    return localIdentifier
}
}

```

- **Background Video Export Function:**

```

Swift
import AVFoundation

class VideoExporter {
    func export(asset: AVAsset, timeRange: CMTimeRange, composition: AVVideoComposition)
async throws -> URL {
        guard let session = AVAssetExportSession(asset: asset, presetName:
AVAssetExportPreset1920x1080) else {
            throw ExportError.sessionCreationFailed
        }

        let outputURL = URL(fileURLWithPath:
NSTemporaryDirectory()).appendingPathComponent(UUID().uuidString).appendingPathExte
nsion("mp4")
    }
}

```



```

    session.outputURL = outputURL
    session.outputFileType = .mp4
    session.timeRange = timeRange
    session.videoComposition = composition
    session.shouldOptimizeForNetworkUse = true

    // Await the modern async export method
    await session.export()

    switch session.status {
    case.completed:
        return outputURL
    case.failed:
        throw session.error ?? ExportError.unknown
    case.cancelled:
        throw ExportError.cancelled
    default:
        throw ExportError.unknown
    }
}
}
}

```

- **SwiftUI View with Conditional Loading UI:**

Swift

```

struct NameMoveView: View {
    @StateObject private var viewModel: AddMoveViewModel
    @State private var elapsedTime: TimeInterval = 0
    private let timer = Timer.publish(every: 1, on:.main, in:.common).autoconnect()

    var body: some View {
        ZStack {
            // Main content of the view (text fields, video preview, etc.)
            mainContentView
                .disabled(viewModel.isSaving)
                .blur(radius: viewModel.isSaving ? 3 : 0)

            if viewModel.isSaving {
                VStack {
                    ProgressView()
                    Text("Saving... \ \(formatTime(elapsedTime))")
                        .padding(.top, 8)
                }
            }
        }
    }
}

```

```

        .padding()
        .background(Color(.systemBackground))
        .cornerRadius(10)
        .shadow(radius: 10)
        .onReceive(timer) { _ in
            elapsedTime += 1
        }
        .onAppear {
            elapsedTime = 0
        }
    }
}
}

// Helper property to check the view model's state
private var isSaving: Bool {
    if case.exporting = viewModel.state { return true }
    if case.savingToLibrary = viewModel.state { return true }
    return false
}
}

```

Verification Through Commutativity Checks (Test Plan)

A comprehensive testing strategy is required to verify that the refined architecture is correct and robust. This involves testing at multiple levels to ensure the system's behavior is predictable and fault-tolerant.

- **Unit Tests:**

- The PhotoKitService should be tested against a mocked PHPhotoLibrary instance to verify the idempotency of getOrCreateAlbum. The test should simulate concurrent calls and confirm that only one creation request is ever triggered.
- The logic for calculating rotation transforms in the VideoExporter should be unit tested with various input angles and video orientations to ensure correctness.

- **Integration Tests:**

- An end-to-end integration test for the "Add Move Flow" is critical. This test will:
 1. Programmatically select a sample video asset.
 2. Set a specific trim range and rotation.
 3. Execute the entire save and persistence flow.

4. After completion, fetch the Move entity from the CoreData store.
 5. Use the stored assetIdentifier to fetch the PHAsset from the actual PHPhotoLibrary.
 6. Request the AVAsset for the saved video and assert that its duration matches the specified trim range and its video track's preferred transform matches the specified rotation.
- A second integration test should run the flow twice in succession to verify that no duplicate album is created.
- **Manual & UI Tests with Profiling Tools:**
 - **Memory Leak Verification:** The most important manual test is to verify the fix for the retain cycle. Use Xcode's **Memory Graph Debugger** to take a snapshot of the memory heap after opening and dismissing the "Add Move Flow." The debugger should show that the TrimmerViewModel and all its related objects have been deallocated.⁹ Running the app with the **Leaks Instrument** in Xcode's Profiler will provide a dynamic analysis and confirm the absence of leaks over time.
 - **Failure Scenario Testing:** Manually test edge cases and failure modes.
 - Deny Photos library permissions and verify that the UI presents a graceful error message.
 - Force-quit the application in the middle of the saving process. Upon relaunch, verify that no empty "BreakDex" album exists in the Photos app.
 - Fill the device's storage to near capacity and attempt a save to verify that storage-related export errors are handled correctly.

Identifying Blind Spots and Future Considerations

Even with a robust architecture, several external factors and future requirements must be considered.

- **iCloud Photo Library:** When a user has iCloud Photo Library enabled, assets are synced across devices. While the localIdentifier is stable, the underlying media data for a newly created asset may not be immediately available on a user's other devices. The UI on those devices must gracefully handle a state where the PHAsset metadata is present but the video data needs to be downloaded from iCloud before playback is possible.
- **Low Device Storage:** The AVAssetExportSession requires temporary disk space to write the new video file. If the device is low on storage, the export will fail. The .failed(Error) state in the state machine must specifically check for AVError.diskFull (or similar errors) and present a clear, actionable message to the user.
- **Video Asset Compatibility:** The system must be robust to various video formats (e.g., HEVC, H.264), containers (.mov, .mp4), and frame rates. Before initiating an export, the

application can check the compatibility of the chosen preset with the source asset using `AVAssetExportSession.determineCompatibility(ofExportPreset:with:outputFileType:completionHandler:)`.

- **Permissions Management:** A user can change application permissions at any time via the Settings app. The `PhotoKitService` must check the `PHPhotoLibrary.authorizationStatus()` before every interaction and handle states like `.denied` or `.restricted` appropriately, guiding the user to re-enable permissions if necessary.
- **Preliminary "Re-link" Functionality Design:** The proposed architecture, which stores the `PHAsset.localIdentifier`, makes future re-linking functionality straightforward. A "Re-link" button in the `MoveDetailView` would trigger a function that takes the `Move` object's `assetIdentifier`. It would then use `PHAsset.fetchAssets(withLocalIdentifiers:options:)` to attempt to retrieve the `PHAsset`.
 - **If found:** The connection is re-established, and the video can be played.
 - **If not found** (because the user deleted the video from their Photos library): The application should present a dialog prompting the user to either select a new video to associate with the move or to delete the `Move` entry from the app's database.

This comprehensive approach, addressing both the immediate bugs and the underlying architectural flaws, will result in a robust, performant, and reliable "Add Move Flow" capable of supporting a large and active user base.

Summary of Architectural Refinements

The following table summarizes the identified issues, their root causes within the categorical model, and the corresponding architectural solutions proposed in this report.

Observed Issue	Symptom(s)	Categorical Failure Model	Root Cause(s)	Proposed Architectural Solution
Memory Leak & Crash	App freezes and is terminated by OS (<code>_pthread_kill</code>) . <code>TrimmerViewModel</code>	Isomorphism between a retain cycle and memory corruption leading to a fatal signal.	Strong reference cycle between <code>TrimmerViewModel</code> and a closure it owns.	Natural Transformation: Apply [weak self] capture list to the offending closure to break the

	deallocated with non-zero retain count.			cycle. Verify deallocation with Memory Graph Debugger.
Empty Photo Albums	Empty albums named "BreakDex" appear in the user's Photos app.	Non-commutative and non-atomic composition of createAlbum and addAsset morphisms.	App crash occurs after album creation but before asset insertion. Lack of idempotency in album creation logic.	Idempotent Functor: Implement a centralized, async, and synchronized PhotoKitService that atomically finds/creates the album and adds the asset in a single performChanges block.
Slow Saving Process	UI becomes unresponsive for an extended period after naming a move.	Inefficient functor (exportVideo) blocking the main thread.	AVAssetExportSession is computationally expensive and is being run on the main thread, blocking all user interaction.	Adjoint Functor (Optimization): Move the entire AVAssetExportSession and PHPhotoLibrary save process to a background DispatchQueue. Use a state machine to publish progress to the UI without blocking it.

Incorrect Saved Video	The final video in the Photos app does not match the trimmed range or rotation shown in the preview.	Incorrect morphism implementation (exportVideo) failing to preserve state.	Improper configuration of AVMutableVideoComposition (for rotation) and/or AVAssetExportSession.timeRange (for trimming).	Morphism Refinement: Correctly configure the video composition with the appropriate CGAffineTransform and ensure the timeRange is precisely passed from the view model to the export session.
Poor User Feedback	User has no indication of progress during the long save; the loading bar after trimming is obtrusive.	Incomplete/suboptimal UX morphisms.	Lack of non-blocking feedback mechanisms for long-running background tasks.	UX Morphism Enhancement: Implement a minimal, non-blocking loading indicator. Use a SwiftUI Timer to display elapsed time, providing continuous and reassuring feedback to the user.
Data Resilience	No clear strategy for handling cases where the link between the app's data and the Photos	Missing object property (localIdentifier) and recovery morphism (relink).	CoreData entity lacks a durable reference to the PHAsset. The source of truth is	Source of Truth Definition: Store the immutable PHAsset.localIdentifier in

	asset is broken.		ambiguous.	CoreData. This establishes the Photos Library as the source of truth and provides a robust mechanism for future re-linking.
--	------------------	--	------------	---

Works cited

1. Check if image exists in Photos - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/36469279/check-if-image-exists-in-photos>
2. Is it possible for dealloc to be called on an object whose retain count is NOT zero?, accessed September 26, 2025, <https://stackoverflow.com/questions/39520663/is-it-possible-for-dealloc-to-be-called-on-an-object-whose-retain-count-is-not-zero>
3. Why would `deinit` be called when retain count is non-zero? - Swift Forums, accessed September 26, 2025, <https://forums.swift.org/t/why-would-deinit-be-called-when-retain-count-is-non-zero/72924>
4. How come retainCount not zero before dealloc gets called? - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/15887380/how-come-retaincount-not-zero-before-dealloc-gets-called>
5. What is a Retain Cycle and What is Not: Memory Management in Swift - Medium, accessed September 26, 2025, <https://medium.com/swift-gurus/what-is-a-retain-cycle-and-what-is-not-memory-management-in-swift-68321140b185>
6. Retain Cycles, Weak and Unowned in Swift - ThomasHanning.com, accessed September 26, 2025, <https://www.thomashanning.com/retain-cycles-weak-unowned-swift/>
7. Understanding Retain Cycles in Swift's ARC: A Simple Guide | Roro.io, accessed September 26, 2025, <https://www.roro.io/post/understanding-retain-cycles-in-swifts-arc-a-simple-guide>
8. Swift closures causing strong retain cycle with self - Stack Overflow, accessed September 26, 2025, <https://stackoverflow.com/questions/50261303/swift-closures-causing-strong-retain-cycle-with-self>
9. Swift Memory Leak: Detecting and Preventing Common Issues in iOS Apps -

- DhiWise, accessed September 26, 2025,
<https://www.dhiwise.com/post/swift-memory-leak-detecting-and-preventing-common-issues>
10. Memory Leaks in SwiftUI and Combine: | by Shashidhar Jagatap | Medium, accessed September 26, 2025,
<https://medium.com/@shashidj206/memory-leaks-in-swiftui-and-combine-e9c35b6fe7b2>
 11. 3 mistakes to avoid with Closures - Swift with Vincent, accessed September 26, 2025,
<https://www.swiftwithvincent.com/blog/three-mistakes-to-avoid-with-closures-in-swift>
 12. How to download & save the video in device storage in swift | by Yasir - Medium, accessed September 26, 2025,
<https://mdcode2021.medium.com/how-to-download-save-the-video-in-device-storage-in-swift-a1129aec736>
 13. How to save image to custom album? - ios - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/28708846/how-to-save-image-to-custom-album>
 14. Requesting Changes to the Photo Library | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/photokit/requesting-changes-to-the-photo-library>
 15. Avoid Create Multiple Album in iOS Swift 5 - Ungapps, accessed September 26, 2025,
<https://www.ungapps.com/2019/10/avoid-create-multiple-album-in-ios.html>
 16. iOS Photo Album Creator | Patrick Spafford, accessed September 26, 2025,
<https://patrickspafford.com/blog/ios-photo-album-creator/>
 17. How to improve AVAssetExportSession saving performance - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/65369762/how-to-improve-avassetexportsession-saving-performance>
 18. AVAssetExportSession | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/avfoundation/avassetexportsession>
 19. Timer | Apple Developer Documentation, accessed September 26, 2025,
<https://developer.apple.com/documentation/foundation/timer>
 20. Timer in SwiftUI - Sarunw, accessed September 26, 2025,
<https://sarunw.com/posts/timer-in-swiftui/>
 21. How do i get the elapsed time of a timer in Swift? - Reddit, accessed September 26, 2025,
https://www.reddit.com/r/swift/comments/rkv32d/how_do_i_get_the_elapsed_time_of_a_timer_in_swift/
 22. Progress indicators | Apple Developer Documentation, accessed September 26, 2025,

<https://developer.apple.com/design/human-interface-guidelines/progress-indicators>

23. Handling loading states within SwiftUI views - Swift by Sundell, accessed September 26, 2025,
<https://www.swiftbysundell.com/articles/handling-loading-states-in-swiftui/>
24. How to add an activity indicator in SwiftUI - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/56496638/how-to-add-an-activity-indicator-in-swiftui>
25. Two ways of capturing self strongly within a closure | Swift by Sundell, accessed September 26, 2025,
<https://www.swiftbysundell.com/articles/two-ways-of-capturing-self-strongly/>
26. ios - PHAssetCollectionChangeRequest: addAssets() now accepts NSFastEnumeration. how to achieve this? - Stack Overflow, accessed September 26, 2025,
<https://stackoverflow.com/questions/30818861/phassetcollectionchangerequest-addassets-now-accepts-nsfasterenumeration-how-to>
27. Making a timer persistant - Using Swift, accessed September 26, 2025,
<https://forums.swift.org/t/making-a-timer-persistant/69677>